N. Shree Deepthi Batch – 37

AI-ASSISTED CODING ASSIGNMENT 21

Task 1 – AI-Assisted Pair Programming

Objective

To understand how AI can assist in generating, improving, and analyzing code.

Task 1 – AI-Assisted Pair Programming (Single Prompt)

Act as an AI pair programmer.

1. First, generate a simple To-Do List Manager program in [Python/Java/JavaScript] with basic features:
   - Add tasks
   - View tasks
   - Delete tasks
   - Use beginner-friendly logic and comments

2. Then, analyze the code and suggest improvements including:
   - Better error handling
   - Additional useful features
   - Code refactoring and optimization
   - Improved user experience

3. Provide an improved version of the program with these enhancements.

4. Finally, compare the original and improved versions by explaining:
   - Key changes made
   - Why they improve the program
   - How AI contributes to better coding and collaboration

Initial Code Generated Using AI

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code 9.py > ...

```python
# A beginner-friendly command-line to-do list manager

tasks = []

def show_menu():
    print("\n=== To-Do List Manager ===")
    print("1. Add task")
    print("2. View tasks")
    print("3. Delete task")
    print("4. Quit")

def add_task():
    task = input("Enter a task to add: ").strip()
    if task:
        tasks.append(task)
        print(f"Added: {task}")
    else:
        print("Empty task not added.")

def view_tasks():
    if not tasks:
        print("No tasks yet.")
        return
    print("\nYour tasks:")
    for i, task in enumerate(tasks, start=1):
        print(f"{i}. {task}")

def delete_task():
    if not tasks:
        print("No tasks to delete.")
        return
    view_tasks()
    idx = input("Enter the task number to delete: ").strip()
    if idx.isdigit():
```

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code 9.py > ...

```python
def delete_task():
    view_tasks()
    idx = input("Enter the task number to delete: ").strip()
    if idx.isdigit():
        idx = int(idx)
        if 1 <= idx <= len(tasks):
            removed = tasks.pop(idx - 1)
            print(f"Deleted: {removed}")
        else:
            print("Invalid task number.")
    else:
        print("Please enter a number.")

def main():
    while True:
        show_menu()
        choice = input("Choose an option: ").strip()

        if choice == "1":
            add_task()
        elif choice == "2":
            view_tasks()
        elif choice == "3":
            delete_task()
        elif choice == "4":
            print("Goodbye!")
            break
        else:
            print("Invalid option. Try again.")

if __name__ == "__main__":
    main()
```

0 ⚠ 0 Update is ready, click to restart. Spaces: 4 UTF-8 { } Python

```
Your tasks:
1. complete work

=== To-Do List Manager ===
1. Add task
3. Delete task
4. Quit
Choose an option: 2

Your tasks:
1. complete work

=== To-Do List Manager ===
1. Add task
1. complete work
```

AI-Suggested Improvements

The AI suggested the following enhancements:

- Added error handling for invalid inputs
- Improved user interaction
- Added extra features (e.g., mark task complete, edit task)
- Refactored code for better readability

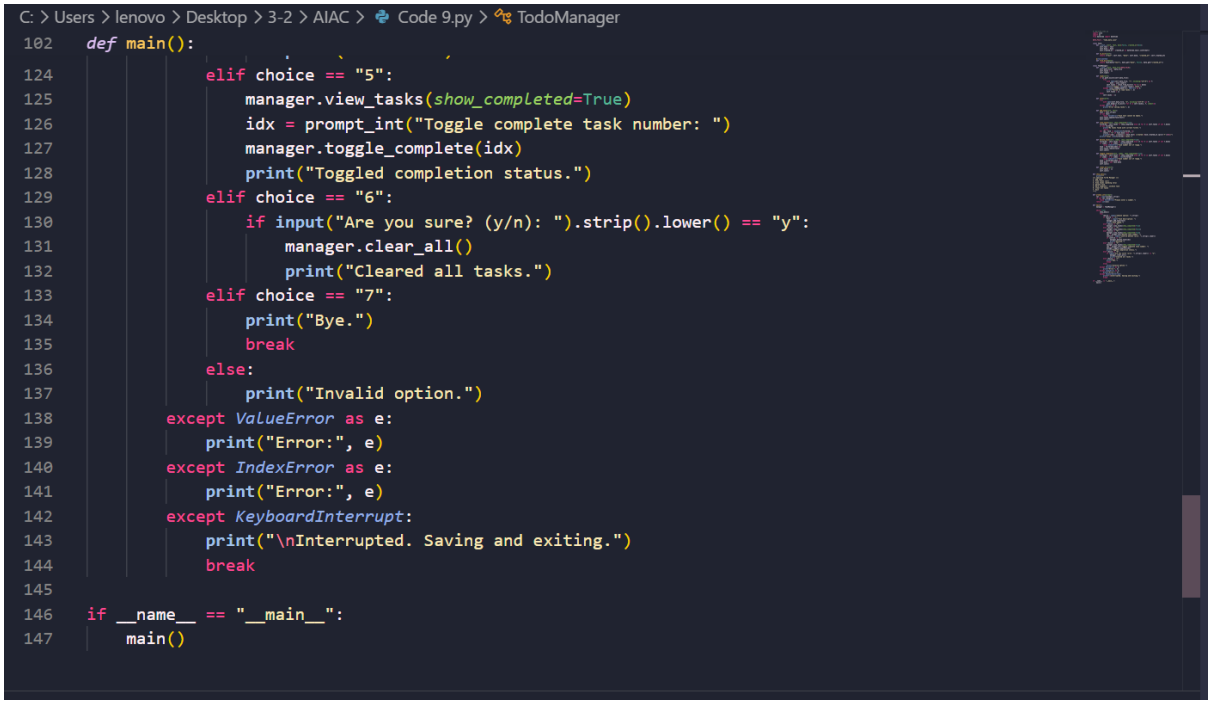
Improved Code

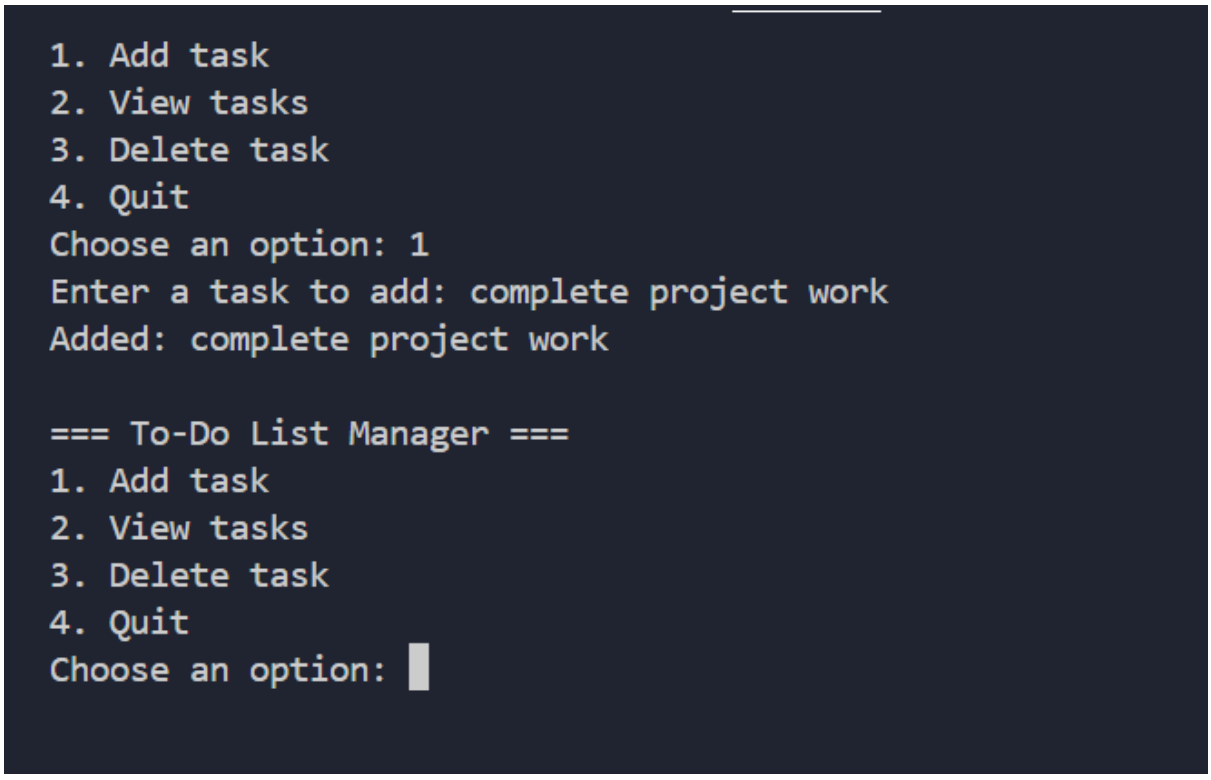
C: > Users > lenovo > Desktop > 3-2 > AIAC > Code 9.py > TodoManager

```python
# improved_todo.py
import json
import os
from datetime import datetime

DATA_FILE = "todo_data.json"

class Task:
    def __init__(self, text, done=False, created_at=None):
        self.text = text
        self.done = done
        self.created_at = created_at or datetime.now().isoformat()

    def to_dict(self):
        return {"text": self.text, "done": self.done, "created_at": self.created_at}

    @staticmethod
    def from_dict(data):
        return Task(data["text"], data.get("done", False), data.get("created_at"))

class TodoManager:
    def __init__(self, data_file=DATA_FILE):
        self.data_file = data_file
        self.tasks = []
        self.load()

    def load(self):
```

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code 9.py > TodoManager

```python
class TodoManager:
    def save(self):
            print("Error saving tasks:", e)

    def add_task(self, text):
        text = text.strip()
        if not text:
            raise ValueError("Task text cannot be empty.")
        self.tasks.append(Task(text))
        self.save()

    def view_tasks(self, show_completed=True):
        filtered = self.tasks if show_completed else [t for t in self.tasks if not t.done]
        if not filtered:
            print("No tasks found with current filter.")
            return
        for idx, task in enumerate(filtered, 1):
            status = "√" if task.done else " "
            print(f"{idx}. [{status}] {task.text} (created {task.created_at.split('T')[0]})")
        print(f"Total {len(filtered)} tasks.")

    def delete_task(self, index, show_completed=True):
        filtered = self.tasks if show_completed else [t for t in self.tasks if not t.done]
        if index < 1 or index > len(filtered):
            raise IndexError("Task number out of range.")
        task = filtered[index-1]
        self.tasks.remove(task)
```





Comparison & Analysis

Aspect	Original Code	Improved Code
Functionality	Basic features	Advanced features added
Error Handling	Minimal	Robust handling

Aspect	Original Code	Improved Code
Code Structure	Simple	Optimized and modular
User Experience	Basic	Improved

Role of AI in Collaboration

AI helped in:

- Quickly generating base code
- Identifying improvements
- Enhancing code quality and efficiency
- Acting as a virtual pair programmer

Task 2 – Version Control Integration

Objective

To use Git for managing and tracking changes in the project.

Task 2 – Version Control Integration (Single Prompt)

Act as a Git and software development expert.

1. Provide step-by-step instructions with commands to:
   - Initialize a Git repository
   - Create and switch to a new branch
   - Add AI-assisted code modifications
   - Commit with meaningful messages
   - Merge changes back into the main branch

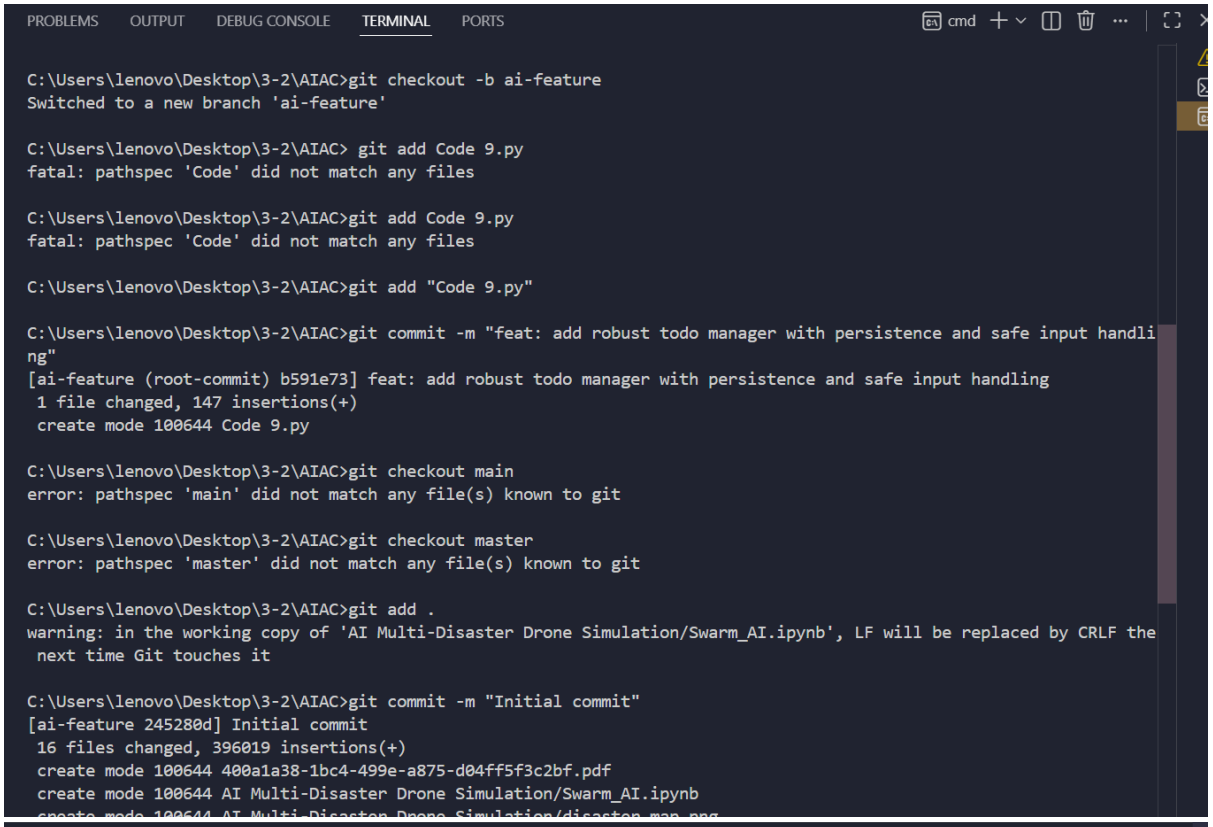
2. Explain each step briefly.

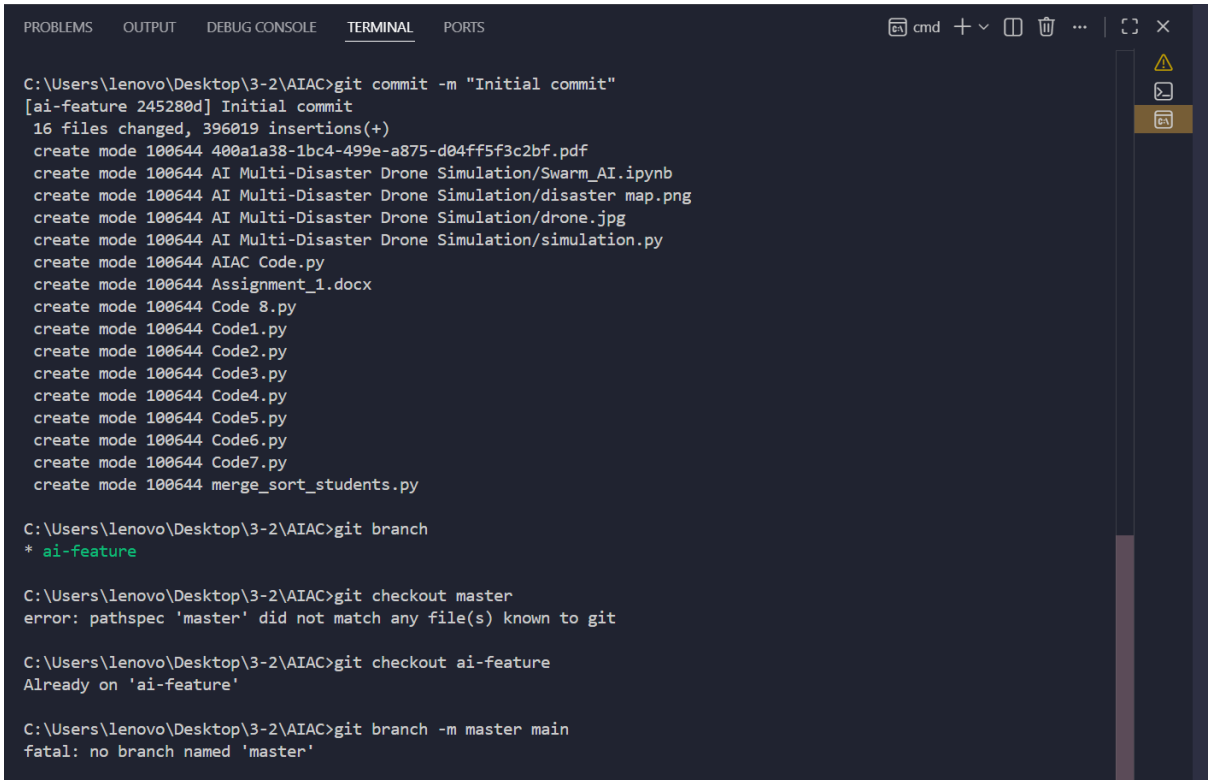
3. Then, reflect on how Git supports AI-assisted development by discussing:
   - Benefits of branching
   - Tracking AI-generated changes
   - Collaboration advantages
   - How version history helps prevent mistakes

Git Commands Used

```
# Initialize repository
git init

# Add files
git add .

# Initial commit
git commit -m "Initial commit"

# Create new branch
git checkout -b ai-feature

# Add AI improvements
git add .
git commit -m "Added AI improvements"

# Switch branch
git checkout main  # or master

# Merge changes
git merge ai-feature
```

Screenshots





Reflection

Version control helps in:

- Tracking AI-generated changes
- Managing different versions of code

- Enabling safe experimentation using branches
- Supporting collaboration and teamwork

Task 3 – AI for Commit Message Generation

Objective

To evaluate AI's role in generating meaningful commit messages.

Task 3 – AI for Commit Message Generation (Single Prompt)

Act as a software engineering assistant.

1. Based on the following code changes:

Generate:
- 3 concise and meaningful Git commit messages
- Follow best practices (clear, short, action-oriented)

2. Then, write 1 human-style commit message for comparison.

3. Compare AI-generated vs human-written messages in terms of:
- Clarity
- Consistency
- Time efficiency

4. Finally, explain how AI-generated commit messages improve:
- Documentation
- Traceability
- Team collaboration

Include practical examples.

AI-Generated Commit Messages

- "Added error handling to To-Do List application"
- "Refactored code for better readability and structure"
- "Implemented additional features for task management"

```
C:\Users\lenovo\Desktop\3-2\AIAC>git commit -m "feat(todo): add JSON persistence to task storage"
On branch ai-feature
```

```
C:\Users\lenovo\Desktop\3-2\AIAC>git commit -m "fix(todo): validate task input and handle invalid delete indexes"
On branch ai-feature
```

Human-Written Commit Message

- "Improved To-Do app with new features and better input handling"

```
C:\Users\lenovo\Desktop\3-2\AIAC>git commit -m "Improve to-do manager by persisting tasks, strengthening input valid
ation, and refactoring core logic to classes
On branch ai-feature
```

Comparison

Criteria	AI Messages	Human Message
Clarity	High	High
Consistency	Very consistent	May vary
Speed	Instant	Takes time

Analysis

AI-generated commit messages:

- Save time
- Maintain consistency
- Improve documentation quality
- Help in better traceability of changes

Task 4 – Pair Programming Simulation

Objective

To simulate collaborative coding using AI assistance.

Task 4 – Pair Programming Simulation (Single Prompt)

Act as two students collaborating on a coding project using AI.

1. Student A:
   - Write a basic version of a program (e.g., To-Do List / Calculator / Expense Tracker)
   - Keep it simple with core functionality and clean code

2. Student B (with AI assistance):
   - Improve the program by:
     - Adding features
     - Enhancing structure
     - Improving user experience
     - Adding error handling
   - Provide the updated code

3. Simulate collaboration using Git by explaining:
   - Branch creation for each student
   - How they worked separately
   - How they merged changes
   - Any conflict resolution (if applicable)

Student A – Initial Code

Code 9.py | # student_a_todo.py Untitled-1

```python
# student_a_todo.py
tasks = []

def add_task():
    task = input("Enter task: ").strip()
    if task:
        tasks.append(task)
        print("Added:", task)
    else:
        print("Task cannot be empty.")

def view_tasks():
    if not tasks:
        print("No tasks yet.")
        return
    print("Tasks:")
    for i, t in enumerate(tasks, 1):
        print(f"{i}. {t}")

def delete_task():
    view_tasks()
    if not tasks:
        return
    idx = input("Task number to delete: ").strip()
    if idx.isdigit() and 1 <= int(idx) <= len(tasks):
        removed = tasks.pop(int(idx) - 1)
        print("Removed:", removed)
    else:
        print("Invalid number.")

def main():
    while True:
        print("\n1=Add 2=View 3=Delete 4=Quit")
        choice = input("> ").strip()
        if choice == "1":
```

0 0 Update is ready, click to restart. Ln 48, Col 11 Spaces: 4

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
3. Delete task
4. Quit
Choose an option: 1
Enter a task to add: AIAC Assignment
Added: AIAC Assignment

=== To-Do List Manager ===
1. Add task
2. View tasks
3. Delete task
4. Quit
Choose an option: 1
Enter a task to add: HPC Assignment
Added: HPC Assignment

=== To-Do List Manager ===
1. Add task
2. View tasks
3. Delete task
4. Quit
Choose an option: 2

Your tasks:
1. complete work
2. complete project work
3. AIAC Assignment
4. HPC Assignment

=== To-Do List Manager ===
1. Add task
2. View tasks
3. Delete task
4. Quit
Choose an option:
```

Student B – AI-Assisted Improvements

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code 10.py > Task > to_dict

```python
import json
import os
from datetime import datetime

DATA_FILE = "todo_data.json"

class Task:
    def __init__(self, text, done=False, created_at=None):
        self.text = text
        self.done = done
        self.created_at = created_at or datetime.now().isoformat()

    def to_dict(self):
        return {"text": self.text, "done": self.done, "created_at": self.created_at}

    @staticmethod
    def from_dict(data):
        return Task(data["text"], data.get("done", False), data.get("created_at"))

class TodoManager:
    def __init__(self, data_file=DATA_FILE):
        self.data_file = data_file
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code 10.py > Task > to_dict

```python
class TodoManager:
    def view_tasks(self, include_done=True):
        filtered = self.tasks if include_done else [t for t in self.tasks if not t.done]
        if not filtered:
            print("No tasks to show.")
            return
        for i, task in enumerate(filtered, 1):
            status = "✓" if task.done else " "
            print(f"{i}. [{status}] {task.text} (created {task.created_at.split('T')[0]})")

    def delete_task(self, idx, include_done=True):
        filtered = self.tasks if include_done else [t for t in self.tasks if not t.done]
        if idx < 1 or idx > len(filtered):
            raise IndexError("Index out of range.")
        task = filtered[idx-1]
        self.tasks.remove(task)
        self.save()

    def toggle_task(self, idx, include_done=True):
        filtered = self.tasks if include_done else [t for t in self.tasks if not t.done]
        if idx < 1 or idx > len(filtered):
            raise IndexError("Index out of range.")
        filtered[idx-1].done = not filtered[idx-1].done
        self.save()

    def edit_task(self, idx, new_text, include_done=True):
        filtered = self.tasks if include_done else [t for t in self.tasks if not t.done]
        if idx < 1 or idx > len(filtered):
            raise IndexError("Index out of range.")
        new_text = new_text.strip()
        if not new_text:
            raise ValueError("Task text cannot be empty.")
        filtered[idx-1].text = new_text
```

```
C: > Users > lenovo > Desktop > 3-2 > AIAC > Code 10.py > Task > to_dict
```

```python

def main():
    todo = TodoManager()
    try:
        while True:
            print_menu()
            choice = input("Option: ").strip()
            try:
                if choice == "1":
                    todo.add_task(input("Task text: "))
                elif choice == "2":
                    todo.view_tasks(True)
                elif choice == "3":
                    todo.view_tasks(False)
                elif choice == "4":
                    todo.view_tasks(True)
                    todo.delete_task(prompt_int("Delete #:"), True)
                elif choice == "5":
                    todo.view_tasks(True)
                    todo.toggle_task(prompt_int("Toggle #:"), True)
                elif choice == "6":
                    todo.view_tasks(True)
                    todo.edit_task(prompt_int("Edit #:"), input("New text: "), True)
                elif choice == "7":
                    if input("Confirm clear all? (y/n): ").lower() == "y":
                        todo.tasks = []
                        todo.save()
                        print("Cleared.")
                elif choice == "8":
                    break
                else:
                    print("Invalid option.")
            except (ValueError, IndexError) as e:
```

```
Choose an option: 1
Enter a task to add: arrays topic revision
Added: arrays topic revision

=== To-Do List Manager ===
1. Add task
2. View tasks
3. Delete task
4. Quit
Choose an option: 2

Your tasks:
1. complete work
2. complete project work
3. AIAC Assignment
4. HPC Assignment
5. arrays topic revision
```

Collaboration Using Git

- Student A created the base program
- Student B created a new branch (ai-feature)
- Improvements were added using AI
- Changes were committed and merged into main branch

Outcome

- Efficient collaboration

- Better code quality
- Faster development

Final Conclusion

This assignment demonstrated the combination of AI and version control systems in modern software development.

Key learnings:

- AI can act as an effective coding assistant
- It improves productivity and code quality
- Git ensures proper tracking and collaboration
- Combining AI with Git enhances the development workflow

Overall, AI-assisted programming along with version control creates a more efficient, collaborative, and structured development process.